P03

GIT (I)

¿QUÉ ES GIT?

- Hoy en día, Git es, con diferencia, el sistema de control de versiones moderno más utilizado del mundo.
- Git es un proyecto de código abierto maduro y con un mantenimiento activo que desarrolló originalmente Linus Torvalds, el famoso creador del kernel del sistema operativo Linux, en 2005.
- Un asombroso número de proyectos de software dependen de Git para el control de versiones, incluidos proyectos comerciales y de código abierto.
- Los desarrolladores que han trabajado con Git cuentan con una buena representación en la base de talentos disponibles para el desarrollo de software, y este sistema funciona a la perfección en una amplia variedad de sistemas operativos e IDE (entornos de desarrollo integrados)

¿QUÉ ES GIT?

- Git, que presenta una arquitectura distribuida, es un ejemplo de DVCS (sistema de control de versiones distribuido, por sus siglas en inglés).

- En lugar de tener un único espacio para todo el historial de versiones del software, como sucede de manera habitual en los sistemas de control de versiones antaño populares, como CVS o Subversion (también conocido como SVN), en Git, la copia de trabajo del código de cada desarrollador es también un repositorio que puede albergar el historial completo de todos los cambios.

- Además de contar con una arquitectura distribuida, Git se ha diseñado teniendo en cuenta el rendimiento, la seguridad y la flexibilidad.

CONCEPTOS BÁSICOS DE GIT

- **Repositorio**: Un repositorio es el elemento más básico de git. Es más fácil imaginarlos como una carpeta del proyecto. Un repositorio contiene todos los archivos del proyecto (incluida la documentación) y almacena el historial de revisión de cada archivo. Los repositorios pueden contar con múltiples colaboradores y pueden ser tanto públicos como privados.

- **Origin** y **main (o master)** son dos términos de uso frecuente cuando se trabaja en un proyecto gestionado con Git. Origin es el nombre por defecto dado a un repositorio remoto, y main es simplemente un nombre de rama.

- **Clonar (clone):** Crea una copia de un repositorio de Git existente. La clonación es la forma más habitual de que los desarrolladores obtengan una copia de trabajo de un repositorio central.

CONCEPTOS BÁSICOS DE GIT

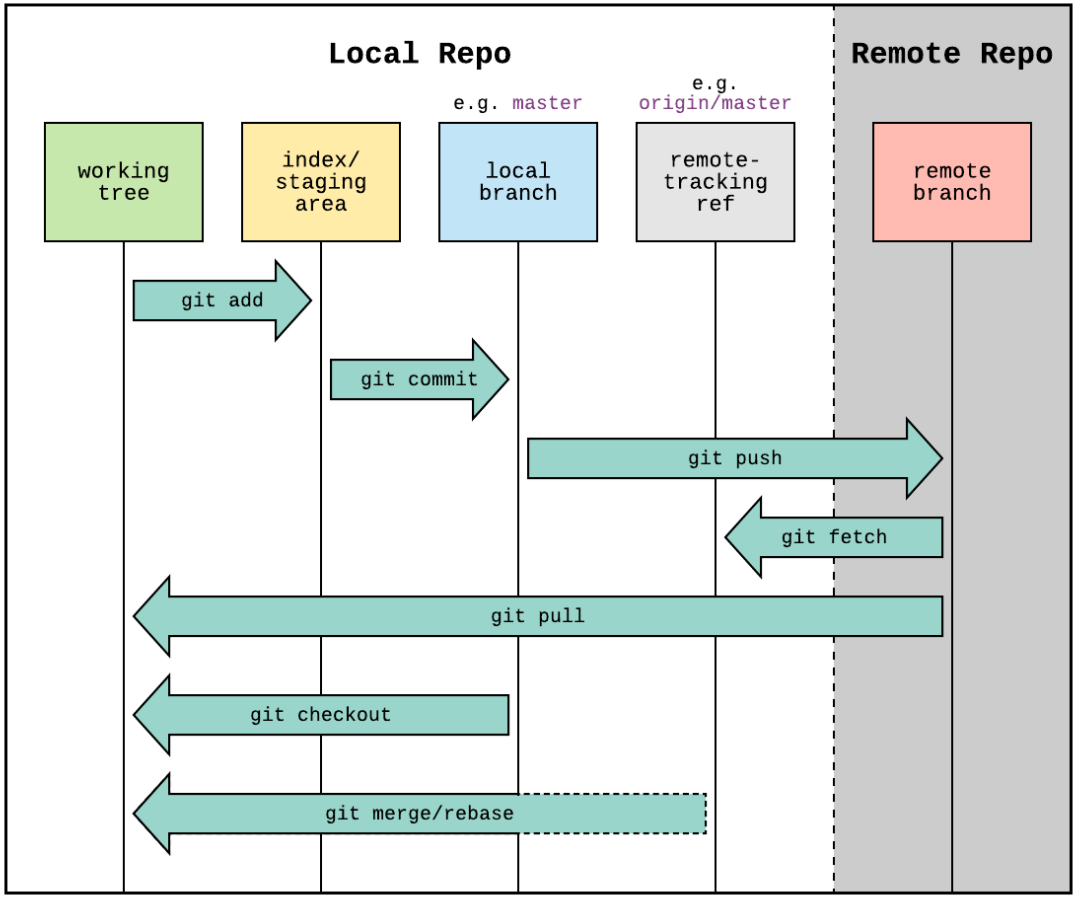
- **Añadir (add):** Mueve los cambios del directorio de trabajo al área del entorno de ensayo. Así puedes preparar una instantánea antes de confirmar en el historial oficial.

- Confirmar (commit): Confirma la instantánea preparada en el historial del proyecto. En combinación con git add, define el flujo de trabajo básico de todos los usuarios de Git.

- **Recuperar (pull):** Descarga una rama de un repositorio remoto e inmediatamente la fusiona en la rama actual.

- **Enviar (push):** Permite mover una o varias ramas a otro repositorio

FLUJO DE TRABAJO DE GIT



GITHUB

- Github es un portal creado para alojar el código de las aplicaciones de cualquier desarrollador, y que fue comprada por Microsoft en junio del 2018.

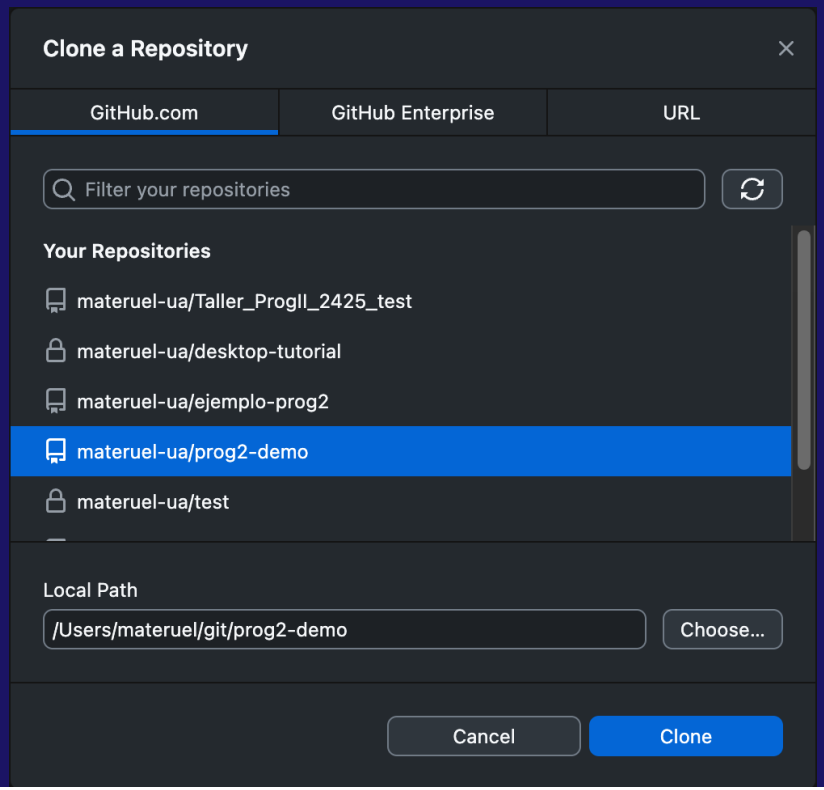
- La plataforma está creada para que los desarrolladores suban el código de sus aplicaciones y herramientas, y que como usuario no sólo puedas descargarte la aplicación, sino también entrar a su perfil para leer sobre ella o colaborar con su desarrollo.

- Vamos a crear una cuenta en GitHub

TAREA 1: USO DE REPOSITÓRIOS

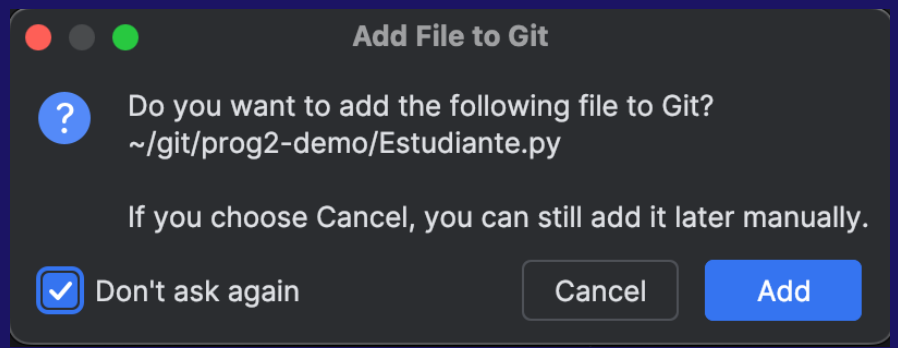
CLONAR REPOSITÓRIOS

1. El profesor creará un repositorio en GitHub y añadirá a los coordinadores de cada grupo
2. Los coordinadores aceptarán la invitación
3. Los coordinadores clonarán el repositorio con GitHub Desktop: File → Clone Repository... → Local Path → Clone:



TAREA

- Abrimos el directorio del repositorio en PyCharm.
- En esta actividad, cada grupo va a trabajar en una clase independiente para construir un sistema de gestión de cursos.
- El coordinador de cada grupo deberá crear en PyCharm un archivo Python con la clase asignada, con todos los atributos solicitados y con un pequeño código que cree y utilice objetos de esa clase.
- Al crear un fichero en Pycham, le diremos que lo añada a git:



TAREA

- Cada grupo creará las siguientes clases en ficheros separados que incluirán código de prueba para crear objetos de cada una de ellas:

  - Grupo 1: Clase Estudiante (estudiante.py)
    - Atributos de clase: total_estudiantes (Contador de instancias de la clase Estudiante)
    - Atributos de instancia: nombre → str, edad → int, cursos_inscritos → list(str)

  - Grupo 2: Clase Profesor (profesor.py)
    - Atributos de clase: total_profesores (Contador de instancias de la clase Profesor)
    - Atributos de instancia: nombre → str, especialidad → str, cursos_asignados → list(str)

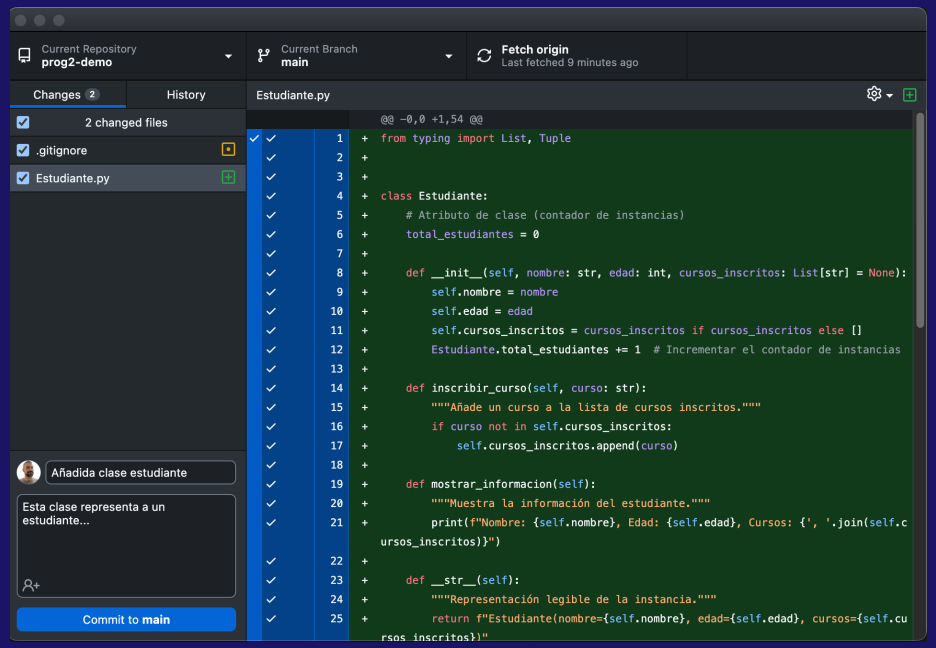
  - Grupo 3: Clase Curso (curso.py)
    - Atributos de clase: total_cursos (Contador de instancias de la clase Curso)
    - Atributos de instancia: nombre → str, codigo → str, descripcion → str

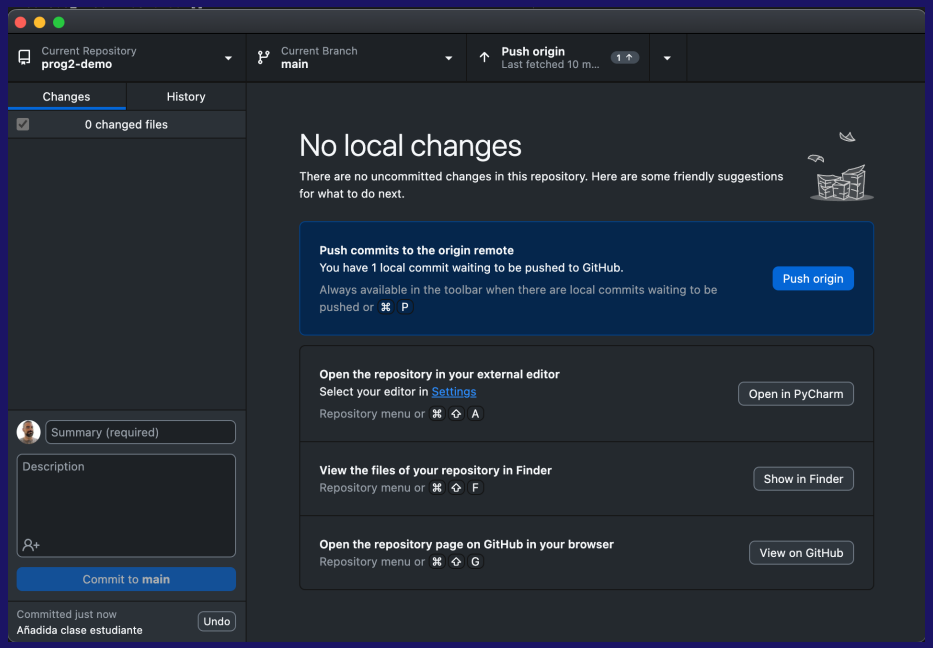
  - Grupo 4: Clase Horario (horario.py)
    - Atributos de clase: total_horarios (Contador de instancias de la clase Horario)
    - Atributos de instancia: curso → str, dia → str, hora_inicio → str, hora_fin → str

  - Grupo 5: Clase Aula (aula.py)
    - Atributos de clase: total_aulas (Contador de instancias de la clase Aula)
    - Atributos de instancia: numero → str, capacidad → int, recursos → list(str)

COMMIT, PUSH Y PULL

1. En GitHub Desktop, añadiremos un resumen y una descripción al commit
2. Haremos commit en main:
3. Haremos push para subir nuestros cambios
4. Cuando todos los equipos acaben, haremos pull para ver su trabajo (Repository → Pull)





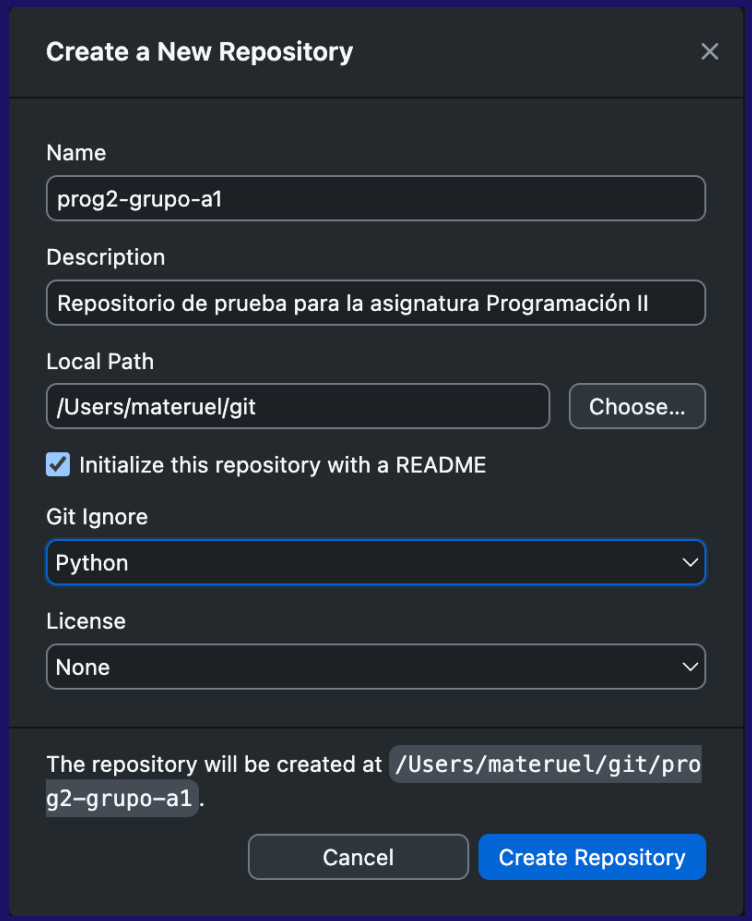
TAREA 2: CREACIÓN DE REPOSITÓRIOS

CREACIÓN DE REPOSITÓRIO

1. El coordinador de cada grupo creará un repositorio en GitHub Desktop (Crear README, Git Ignore Python)

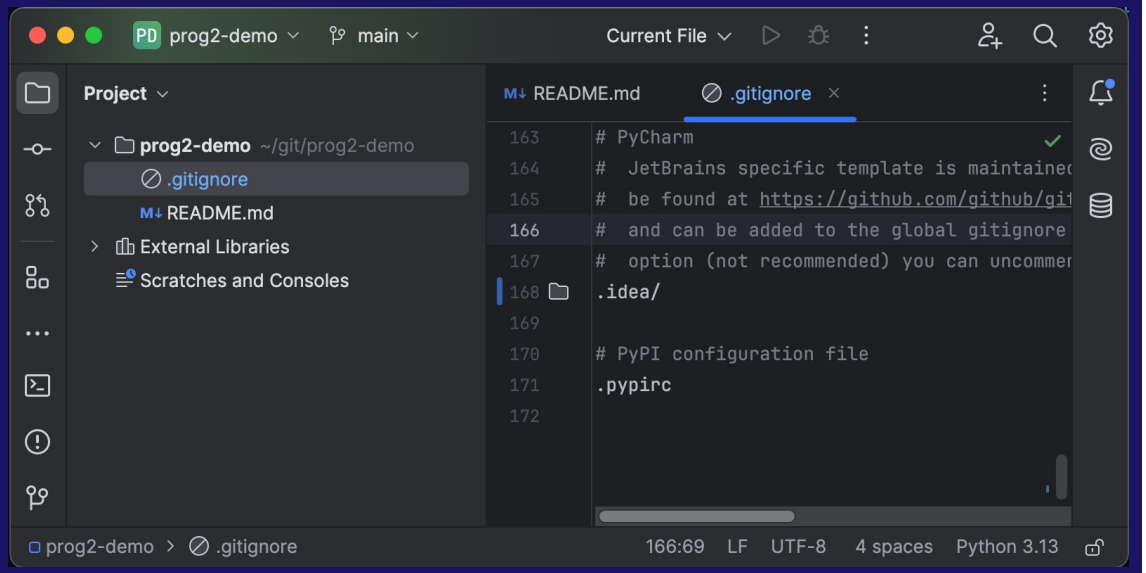
2. El coordinador invitará al repositorio a los integrantes del equipo: Web GitHub del repositorio → Settings → Collaborators

3. Los integrantes del equipo aceptarán la invitación (desde Web GitHub) y clonarán el repositorio



GITIGNORE

- El archivo .gitignore es un archivo de configuración en Git que especifica qué archivos o carpetas deben ser ignorados por el control de versiones.
- Se usa para evitar que archivos innecesarios, como configuraciones locales, archivos temporales o dependencias generadas automáticamente, sean incluidos en el repositorio.
- El coordinador del grupo descomentará la línea #.idea/ para que git no guarde la configuración de PyCharm y hará un commit y un push-
- El resto de integrantes harán un pull para obtener el nuevo .gitignore
- En el futuro, también usaremos .gitignore para evitar guardar los VENV



TAREA

1. El coordinador creará el fichero enciclopedia.py, hará commit y push para subir el fichero a GitHub:

```python
def obtener_definicion(tema):
    definiciones = {
        # Aquí cada persona agregará su tema y definición
    }
    return definiciones.get(tema.lower(), "Definición no encontrada")

if __name__ == "__main__":
    tema = input("Ingrese un tema: ")
    print(obtener_definicion(tema))
```

2. El resto de integrantes añadirán (tras hacer pull), definiciones a la enciclopedia hará commit y push. Por ejemplo:

```python
definiciones = {
    "python": "Un lenguaje de programación versátil y fácil de aprender.",
    "git": "Un sistema de control de versiones distribuido.",
}
```

3. Si push da un error por conflictos, haced pull antes y fusionar vuestros cambios